Report Assignment # 4:
(Code Review – Soumil Makhija – James Hoang)

The following code smells were identified and refactored by Soumil:

1. Hard-coded Resource Path

Replaced hard-coded paths with named constants to improve readability and simplify future changes.

2. Low Cohesion

Split large rendering method into smaller focused helpers for easier reading and testing.

3. Manual Array Copying

Replaced complex System.arraycopy calls with simpler clone operations for cleaner maintainable code.

4. Repeated Drawing Logic

Centralized duplicated canvas drawing logic into single reusable method to reduce code duplication.

5. Long Method

Split lengthy stat update method into five focused helpers to improve clarity and cohesion.

6. Unclear Coupling

Extracted duplicated canvas dimension logic into dedicated helper methods to improve code clarity.

7. Unnecessary Null Check

Removed redundant null checks on FXML fields that cannot be null when properly configured.

8. Long Method

Refactored lengthy config update method into smaller focused helpers for better readability and maintainability.

The following code smells were identified and refactored by James:

9. Bad Naming

Renamed unclear variables and methods to better reflect their purpose and domain meaning.

10. Dead Code & Unused Variables

Removed unused fields and methods throughout the codebase, reducing clutter and potential confusion.

11. High Coupling

Introduced value objects to encapsulate related data and reduce direct dependencies on constants.

12. Data Clump

Replaced repeated coordinate pairs with a Position value object, improving method signatures.

13. Low Cohesion

Extracted medkit logic into dedicated MedkitInventory class, ensuring single responsibility per class.

14. Lack of Documentation

Added comprehensive Javadoc comments to all public classes and methods for better understanding.

15. Unjustified Primitives

Replaced primitive values with domain-specific value objects, adding type safety and semantic meaning.

16. Primitive Obsession

Consistently used value objects to represent domain concepts instead of primitives throughout codebase.